

Stealthy Computational Delay Attacks on Control Systems

Talitha Nauta
talitha.nauta@control.lth.se
Lund University
Sweden

Henrik Sandberg
hsan@kth.se
KTH Royal Institute of Technology
Sweden

Martina Maggio
maggio@cs.uni-saarland.de
Saarland University, Germany
Lund University, Sweden

SYNOPSIS

Cyber-Physical Systems (CPS) are integral to critical infrastructure, but their interconnected nature exposes them to sophisticated cyber threats. Traditional security mechanisms primarily focus on detecting direct manipulations of control signals or sensor data, leaving them vulnerable to more subtle attack vectors. This paper introduces a novel optimisation framework for modelling and executing stealthy computational delay attacks—an attack class that subtly interferes with controller execution timing to degrade system performance while remaining undetected. Unlike conventional denial-of-service attacks, these stealthy attacks introduce delays selectively, ensuring that the controller fails to compute control signals in time without triggering standard anomaly detection mechanisms.

We formulate the problem as a Mixed Integer Quadratically Constrained Programming (MIQCP) optimisation, allowing attackers to maximise system disruption while evading detection. Our framework is evaluated on two control systems: a simulated stable quadruple-tank process and a real-hardware implementation of an unstable Furuta pendulum. Experimental results demonstrate that even brief, undetected computational delays can lead to severe performance degradation and system destabilisation. These findings highlight the need for improved intrusion detection mechanisms that account for time-based threats, emphasising long-term activity monitoring and adaptive defence strategies to safeguard CPS integrity.

CCS CONCEPTS

• **Security and privacy** → *Intrusion detection systems*; • **Theory of computation** → **Stochastic control and optimization**; • **Information systems** → **Process control systems**.

KEYWORDS

Security, Control Systems, Computational Delay Attacks

ACM Reference Format:
Talitha Nauta, Henrik Sandberg, and Martina Maggio. 2025. Stealthy Computational Delay Attacks on Control Systems. In *ACM/IEEE 16th International Conference on Cyber-Physical Systems (with CPS-IoT Week 2025) (ICCPS '25), May 6–9, 2025, Irvine, CA, USA*. ACM, New York, NY, USA, 11 pages. https://doi.org/10.1145/3716550.3722013



This work is licensed under a Creative Commons Attribution 4.0 International License.
ACM/IEEE 16th International Conference on Cyber-Physical Systems (with CPS-IoT Week 2025) (ICCPS '25), May 6–9, 2025, Irvine, CA, USA, 2025.
© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1498-6/2025/05
https://doi.org/10.1145/3716550.3722013

1 INTRODUCTION

Cyber-Physical Systems (CPS) are increasingly crucial to critical infrastructure and industrial applications, yet their interconnected nature makes them highly vulnerable to cyber attacks [5, 38]. These systems integrate both computational and physical components, ensuring precision in industries such as manufacturing, energy, transportation, and healthcare. However, due to their reliance on real-time data and control loops, they are susceptible to sophisticated cyber threats. Attackers can manipulate system functionality in subtle ways that evade traditional detection mechanisms.

Historical cyber attacks, such as the 2010 Stuxnet attack and the 2014 German steel mill attack, have demonstrated the catastrophic potential of cyber intrusions that manipulate input signals or output measurements [12, 39]. These incidents highlight the urgent need for advanced defence mechanisms to counter emerging attack vectors that, while less perceptible, can be equally disruptive. One such emerging threat is the stealthy computational delay attack [8, 20], a time-based intrusion that selectively introduces artificial delays in the computation of control signals. Unlike traditional Denial-of-Service (DoS) attacks, which flood a system and are more easily detected, computational delay attacks operate within normal system fluctuations, making them significantly harder to identify using conventional monitoring techniques [21, 46].

This paper focuses on the modelling and evaluation of stealthy computational delay attacks. While we discuss natural detection mechanisms used in industry, our primary contribution lies in demonstrating the effectiveness and stealthiness of these attacks, emphasising the need for improved defences.

Our approach systematically analyses key CPS components, including the controlled physical plant, the discrete-time controller, and the detection mechanisms designed to identify anomalies in system timing. We introduce a novel optimisation-based attack framework that enables an adversary to systematically maximise system disruption while evading detection. Our framework ensures stealthiness by incorporating statistical constraints based on binomial and geometric distribution tests, allowing the attack to remain undetected under conventional activity-based monitoring. We formulate the attack as a Mixed Integer Quadratically Constrained Programming (MIQCP) problem, solvable with state-of-the-art optimisation tools such as `Gurobi` [9].

To assess the real-world feasibility and impact of these attacks, we evaluate them on two distinct control systems: (i) a stable system modelled as a quadruple-tank process [10], where the attack subtly disrupts setpoint regulation; and (ii) an unstable system, represented by a Furuta pendulum [49], where even brief delays can lead to destabilisation. Through extensive simulations and real-hardware experiments, we demonstrate that the success of computational delay attacks is influenced by multiple factors, including: (i) the system's tolerance to consecutive deadline misses, (ii) the length of

the detection window, and (iii) the actuation strategy used when a deadline is missed. Ultimately, our findings underscore the urgent need for more robust detection mechanisms that incorporate long-term activity monitoring and adaptive defence models. By exposing the vulnerabilities of modern CPS to stealthy computational delay attacks, this work provides crucial insights for the development of more resilient control system security strategies.

The remainder of this paper is organised as follows: Section 2 presents the attack in context and the related research, Section 3 presents the system and detection model, Section 4 presents the attack characteristics and the optimisation problem that the attacker should solve, Section 5 shows our experimental evaluation, and Section 6 concludes the paper.

2 RELATED WORK

Cyber-Physical Systems (CPS) have become increasingly targeted by cyber attacks in recent years. To effectively safeguard CPSs, a comprehensive security framework is essential—one that not only classifies different attack types but also considers the resources required by adversaries, such as system knowledge, data disclosure, and disruption capabilities.

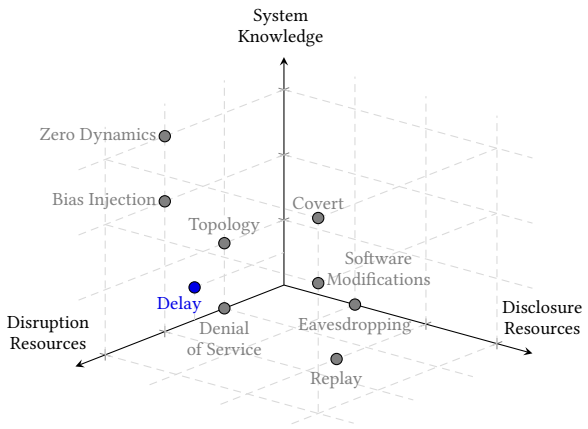


Figure 1: Taxonomy of known attacks against cyber-physical systems, source: [38]. The taxonomy is here extended introducing the *delay* attack.

A well-established taxonomy proposed in [38] categorises attack types based on their impact on physical processes, highlighting how adversaries exploit specific vulnerabilities. This taxonomy has been further refined in [5], which introduces a risk-based evaluation framework that assesses attacks along three dimensions: (i) the attack scenario (how the attack is executed), (ii) the potential impact on the physical system, (iii) the likelihood of attack success given the system's defences. From a control-theoretic perspective, these studies emphasise the importance of securing CPSs against a wide range of threats, including data manipulation, denial-of-service attacks, and timing-based system disturbances that bridge cyber vulnerabilities with physical consequences.

This work extends the taxonomy [38] by incorporating a new category of attacks—computational delay attacks—which exploit timing vulnerabilities in control systems. Figure 1 presents this extended taxonomy.

2.1 Categories of CPS Attacks

In the following we briefly review the categories of CPS attacks and their relation to computational delay attacks.

– **False-Data Injection Attacks:** False-data injection attacks involve modifying sensor or actuator data to mislead the control system, potentially causing incorrect decision-making and operational failures [11, 13, 28]. Different classes of these attacks vary based on the attacker's access to system components:

- Bias Attacks: Introduce a constant offset in data, leading to gradual system drift and performance degradation [38].
- Zero-Dynamics Attacks: Exploit knowledge of the physical system model to induce instability without triggering alarms [29].
- Covert Attacks: Manipulate data while maintaining the illusion of normalcy, evading traditional detection mechanisms [12, 14, 34].
- Replay Attacks: Intercept and resend previously recorded valid data, deceiving the system into misinterpreting outdated information as real-time input [27].

– **Eavesdropping and Denial-of-Service Attacks:** Eavesdropping attacks passively intercept communication to gather sensitive system information, such as control commands or passwords, without directly altering system behaviour [5]. Denial-of-Service (DoS) attacks, in contrast, actively overwhelm the system by flooding it with excessive requests, rendering it unresponsive [4]. While DoS attacks and delay-based attacks can both disrupt CPS functionality by interfering with control loops, DoS attacks typically cause large, easily detectable disruptions, whereas computational delay attacks operate within normal system fluctuations, making them significantly harder to detect [1, 6, 43].

– **Topology and Software Modification Attacks:** Recent research has introduced two additional attack types not covered in the original CPS attack taxonomy [38]:

- Topology Attacks maliciously alter network configurations, such as falsely reporting the status of nodes, links, or devices, leading to communication disruptions [45].
- Software Modification Attacks tamper with system software, including firmware and control algorithms, to introduce vulnerabilities and backdoors, affecting security, performance, and functionality [31].

2.2 Computational Delay Attacks

Computational delay attacks are a category of timing-based attacks that exploit the control system's dependence on precise timing. Unlike data injection or DoS attacks, which focus on altering information flow or system availability, computational delay attacks disrupt the execution of control tasks by selectively delaying the computation of control signals [44].

Early studies primarily examined delay attacks affecting communication channels rather than control computations [3, 22]. Continuous-time delay attacks have been analysed in [41, 42], where constant delays in feedback loops were shown to degrade system performance. Time-varying delays affecting discrete-time

communication channels were explored in [16], highlighting their disruptive potential.

However, our work focuses on delays in control computation rather than transmission, a distinct attack vector that does not rely on disclosure resources (i.e., the attacker does not need to know system states or sensor values). Instead, an attacker can execute central processing unit (CPU) starvation attacks, blocking the control task from completing within its allocated time window.

Recent research [20, 21] introduced methods for detecting execution timing anomalies, but their ability to prevent stealthy computational delay attacks remains limited. For control systems, [8] demonstrated that such attacks can successfully degrade system performance, but the full extent of their impact, especially under active defence mechanisms, has not been comprehensively analysed. This paper fills that gap by:

- Evaluating the effectiveness of computational delay attacks on both stable and unstable systems.
- Developing an optimisation-based attack model to maximise system disruption while remaining undetected.
- Assessing the ability of statistical defence mechanisms (e.g., binomial and geometric distribution tests) to mitigate such attacks.

By analysing these attacks within a rigorous mathematical framework, this work provides a deeper understanding of how computational delay attacks operate, how they can evade detection, and how to enhance CPS security against them.

3 SYSTEM

To better understand the impact of computational delay attacks, we define a mathematical model of the targeted control systems. Specifically, in Section 3.1 we introduce the system model and in Section 3.2 we introduce classical defences used to detect an ongoing attack.

3.1 Plant and control model

We consider a discrete-time linear time-invariant plant $\mathcal{P}$ that models a physical system controlled by a linear controller C [18, 48].

Definition 3.1 (System quantities). We denote with $x_k \in \mathbb{R}^{n_x}$ the state of the physical plant, with $y_k \in \mathbb{R}^{n_y}$ the output of the physical plant, with $u_k \in \mathbb{R}^{n_u}$ the input of the physical plant, and with $z_k \in \mathbb{R}^{n_z}$ the state of the controller at the beginning of the k-th sampling period.

The model of the physical system can be written as follows,

$$\mathcal{P} : \begin{cases} x_{k+1} = A\,x_k + B\,u_k \\ y_k = C\,x_k + D\,u_k \end{cases}, \tag{1}$$

where the matrices $A \in \mathbb{R}^{n_x \times n_x}$, $B \in \mathbb{R}^{n_x \times n_u}$, $C \in \mathbb{R}^{n_y \times n_x}$, and $D \in \mathbb{R}^{n_y \times n_u}$ are constants and depend on the physics. The controller computes a control signal for the next period according to

$$C : \begin{cases} z_{k+1} = F\,z_k + G\,y_k \\ u_{k+1} = H\,z_k + K\,y_k \end{cases}, \tag{2}$$

where the matrices $F \in \mathbb{R}^{n_z \times n_z}$, $G \in \mathbb{R}^{n_z \times n_y}$, $H \in \mathbb{R}^{n_u \times n_z}$, and $K \in \mathbb{R}^{n_u \times n_y}$ are the result of the control design process.

At the beginning of the k-th sampling period, the plant receives the vector u_k as input that is kept constant during the entire period. The vector y_k indicates the output of the physical plant, which depends on the internal state x_k. Following the logical execution time paradigm (or the one-step delay design paradigm in control terms), the controller senses the value of y_k and during the k-th iteration produces the control signal u_{k+1} that will be applied to the plant in the following period. The calculation of u_{k+1} depends on the internal controller state z_k, which is updated in every iteration. The controller executes these operations periodically, and one iteration of this loop is referred to as a *job* of the control task. Without loss of generality, we assume that the controller has the objective to steer the output to 0.

In our model, the control task uses the last known measurement instead of a fresh state estimate. This assumption is based on the fact that the control task is allowed to continue in the event of a deadline miss, which is the most common control implementation [24]. Even in systems where a separate state estimator is present, computational delays often result in stale state estimates due to scheduling constraints. While some CPS architectures could mitigate this by prioritising state updates, our approach aligns with widely used control paradigms where controllers operate on buffered or sampled measurements.

3.2 Attack detector model

As a response to the growth of attacks on control systems, many systems now employ detection mechanisms that aim at discovering that the control system is under attack. These detection mechanisms can broadly be classified into two distinct groups: *physics-based* and *activity-based*.

Physics-based detection systems rely on the knowledge of the physical system model (i.e., on the matrices A, B, C, and D) and the controller model (i.e., on the matrices F, G, H, and K) to detect attacks [15, 26, 29, 37–39]. These detectors are based on the idea that the physical system, albeit subject to noise and disturbance, should behave roughly according to the model, and any deviation from the model that is sustained over time is an indication of an ongoing attack. These systems are effective in detecting attacks that manipulate sensor and actuator data, as the detector can check that the measured output of the system (i.e., y) and the applied control signal (i.e., u) are consistent with estimates of what they should be if no attack were being carried out. Attackers can sometimes evade these detectors by carefully manipulating the system to make the attack look like a disturbance, or by manipulating the system in a way that is consistent with the model, crafting attacks that are typically referred to as *stealthy* [36]. Furthermore, if the attack delays the execution of the controller, the physical model and the actual actions are consistent, making this detection mechanism ineffective against delay attacks.

Activity-based detection systems, on the other hand, monitor the computational environment and system activity (e.g., memory usage, network traffic, timing of the control task execution) to identify potential attacks [7, 23, 32, 40]. These intrusion detectors are ineffective against attacks that manipulate the control signal or sensor data, but can identify an attacker causing computational delays. We therefore consider that such a detection mechanism is

in place[1] and that the attacker aims at both disrupting the system behaviour as well as avoiding detection.

We assume that the detector monitors the timing of the control task in the operating system and detects if the control signal is produced on time (deadline hit) or if the computation is not completed by the time the new control signal should be applied (deadline miss). The detector keeps track of the controller execution using two different information sources: a sliding window with the last executions and historical data.

Definition 3.2 (Sliding window $\mathcal{W}$). $\mathcal{W} = (w_i)_{i\in\{1,\dots,n_w\}}$ denotes a sliding window that tracks anomalies in the last n_w controller task executions, with $w_i \in \{0, 1\}$. Specifically, $w_i = 0$ records the absence of anomalies and $w_i = 1$ indicates that an anomaly caused a deadline miss at the i-th iteration. The value of w_{n_w} always represents the most recent iteration.

Definition 3.3 (Long-term counters $\mathcal{M}$). $\mathcal{M} = \{m_0, ..., m_i, ...\}$ introduces a set of counters that track the number of deadline misses from the beginning of the controller execution. The value of m_i represents the number of occurrences of i consecutive deadline misses followed by a deadline hit experienced by the control task since its initialisation.

These two information sources are chosen due to their simplicity and effectiveness. The aim of the choice is to balance memory consumption and available knowledge. Specifically, one can use n_w to adjust to the amount of memory. Long-term data can instead be used to check that rare events are indeed happening only on very few occasions.

4 ATTACK

In this section, we present the details of the proposed attack and show its stealthiness. The attack involves the introduction of artificial delays in the calculation of the control signal, which causes deadline misses.

4.1 Attack model and realisation

The attack is realised as a central processing unit starvation attack, as discussed in the literature [2, 8, 19–21, 25]. The form we analyse is the following: an attacker waits for the triggering of the control task execution and starts a (malicious) task that takes over the processing units for a prescribed amount of time. Usually, this task has higher priority than the controller task, but recent studies demonstrated that this could also be achieved with lower priority tasks [20, Section III]. The scheduler suspends the execution of the control task and executes the malicious task instead.

In addition to the detection mechanism that may identify the ongoing attack (introduced in Section 3.2), typically control systems embed basic protection mechanisms. The simplest and most common of these mechanisms is a watchdog that shuts down the control system after it experiences a given number of consecutive deadline misses. While the exact threshold may vary across systems, attackers can infer it through side-channel analysis, observing system behaviour under normal and degraded conditions, or leveraging knowledge of industry best practices. Watchdog thresholds in industrial control systems typically range between 3 and 10 missed deadlines, providing an empirical basis for our attack model.

Definition 4.1 (Maximum consecutive deadline misses $\bar{a}$). We denote with $\bar{a}$ the maximum number of deadline misses that the attacker can introduce in the execution of the control task without triggering a basic system shutdown. As a result, the maximum experienced delay between sensing and actuation is $\bar{a} + 1$ periods.

We assume that a control job that experiences a deadline miss is continued at the deadline, taking the place of the following job, according to common practice in industry [24, 30]. However, no new control signal is calculated. Depending on the controller implementation, the control signal may be kept constant, following a *hold* policy, or set to zero following a *zero* policy [33]. Both these alternatives are used in practice, and the choice depends on the specific application and the physical system that is controlled, hence we consider both in our attack definition.

Attack model. At the activation of each job of the control task, the attacker may choose to alter the job's *response time*, introducing a computational delay in control signal calculation and causing a given number of *deadline misses*.

Defence model. The detector monitors the controller execution, recording deadline misses using a sliding window and long-term counters. If too many consecutive deadlines are missed, the detector performs a plant shutdown.

4.2 Attack effect

We now consider the effect of the attack on the controller.

Definition 4.2 (Latest known measurement $\tilde{y}$). We denote with $\tilde{y}_k$ the latest known measurement of the output of the physical system at time k. The latest known measurement is sensed at the beginning of the controller task execution and is not updated in case of a deadline miss.

As a consequence of the attack, the control task misses a deadlines and completes the calculation of the next control signal in the $(a + 1)$-th period. The computation is then completed and uses the last known measurement $\tilde{y}_k$ available at time k. The controller model presented in Equation (2) is then modified as follows,

$$C_a : \begin{cases} z_{k+i} = z_k & \forall\{i \in \mathbb{N} \mid 1 \le i \le a\} \\ z_{k+a+1} = F\, z_k + G\, \tilde{y}_k & \\ u_{k+i} = \Delta\, u_k & \forall\{i \in \mathbb{N} \mid 1 \le i \le a\} \\ u_{k+a+1} = H\, z_k + K\, \tilde{y}_k & \end{cases} \tag{3}$$

where Δ is the identity matrix of size n_u if the controller is using the hold actuation strategy, and a zero matrix if the controller is using the zero strategy. The equation is simply stating that the controller state is kept constant, while the control signal is updated (either keeping it to the same value or setting it to zero) at every iteration in which there is a deadline miss. Finally, when the computation completes, it does so using the last known value of the output $\tilde{y}$. In order to analyse the attack impact, we need to model precisely

[1] The detector executes either on the same machine as the controller, without disruption, or on a machine that directly communicates with it without the communication channel being compromised.

the effect of the attack on the system state. Hence, we define the closed-loop system state ψ, which includes the controller and the physics.

Definition 4.3 (Closed-loop state ψ). We denote with ψ_k the state of the closed-loop system at time k. Specifically $\psi_k = [x_k^\top \ \tilde{y}_k^\top \ z_k^\top \ u_k^\top]^\top$, where $(\cdot)^\top$ is equal to the transpose of $(\cdot)$. The signals u_k and $\tilde{y}_k$ are introduced in the closed-loop state in order to analyse the effect of the computational delay. The size of the closed-loop state vector is $n_\psi = n_x + n_y + n_z + n_u$.

We now determine the evolution of the closed-loop state when the controller completes its execution on time or not. The closed-loop state always evolves according to the following equation,

$$\psi_{k+1} = [\Phi_\text{h} \mid \Phi_\text{m}]\, \psi_k, \tag{4}$$

meaning the system evolves either according to Φ_h or according to Φ_m, with the two matrices defined as follows.

Definition 4.4 (Closed-loop state matrix with a deadline hit Φ_h). The closed-loop system matrix Φ_h determines the evolution of the system in case of deadline hit. Said evolution can be written as

$$\psi_{k+1} = \begin{bmatrix} x_{k+1} \\ \tilde{y}_{k+1} \\ z_{k+1} \\ u_{k+1} \end{bmatrix} = \begin{bmatrix} A & 0 & 0 & B \\ CA & DK & DH & CB \\ 0 & G & F & 0 \\ 0 & K & H & 0 \end{bmatrix} \begin{bmatrix} x_k \\ \tilde{y}_k \\ z_k \\ u_k \end{bmatrix} \equiv \Phi_\text{h}\, \psi_k, \tag{5}$$

hence defining the matrix Φ_h.

Definition 4.5 (Closed-loop state matrix with a deadline miss Φ_m). The closed-loop system matrix Φ_m determines the evolution of the system in case of deadline miss. Said evolution can be written as

$$\psi_{k+1} = \begin{bmatrix} x_{k+1} \\ \tilde{y}_{k+1} \\ z_{k+1} \\ u_{k+1} \end{bmatrix} = \begin{bmatrix} A & 0 & 0 & B \\ 0 & I_{n_y} & 0 & 0 \\ 0 & 0 & I_{n_z} & 0 \\ 0 & 0 & 0 & \Delta \end{bmatrix} \begin{bmatrix} x_k \\ \tilde{y}_k \\ z_k \\ u_k \end{bmatrix} \equiv \Phi_\text{m}\, \psi_k, \tag{6}$$

hence defining the matrix Φ_m.

Note that we use I_n to indicate the identity matrix of size n and recall that $\Delta = I_{n_u}$ if the controller is using the hold policy, and a zero matrix of the same size if the controller is zeroing the control signal.

4.3 Maximum impact

We now discuss what the *worst-case* attack is that the attacker can perform, while remaining undetected. Here worst-case is intended with respect to a cost function that corresponds to the objective of the attacker. As the attacker wants the attack to be stealthy, they cannot disrupt every single execution of the controller task (Definition 4.1) and should rather focus on specific iterations. To understand what is the worst-case scenario, we formulate an optimisation problem to determine an optimal attack signal. We now introduce the ingredients of said optimisation problem.

Definition 4.6 (Attack length ℓ). We denote with ℓ the attack duration, expressed as the number of control task periods for which the attack is optimised. Using a specific attack length does not exclude the possibility of repeating the attack, even though this may impact the attack stealthiness.

The objective function of the optimisation problem is to maximise the displacement of the output compared to where the output vector would be if the plant were controlled normally. As the controller is trying to regulate the output to zero, said objective can be formulated as $y^\top Q\, y$, where $Q \in \mathbb{R}^{n_y \times n_y}$ is a positive semi- definite matrix that allows the attacker to specify a weight for the different components of the output signal y. Specifically, the attacker uses the entire attack length to build up the effect of the attack,[2] therefore the optimisation problem aims at maximising $y_\ell^\top\, Q\, y_\ell$.

Definition 4.7 (Attack $(a_i)_{i\in\{1,\dots,\ell\}}$). An attack sequence $(a_i)_{i=1}^{\ell}$ is a sequence of ℓ binary values, where $a_i = 0$ implies no attack in the i-th iteration, while $a_i = 1$ implies that the attacker forces a deadline miss for the i-th controller period.

With the given definitions, we can write the optimisation problem that determines the attack that has the most disruptive impact on the plant output, while remaining undetected:

$$\begin{aligned} (a_i^*)_{i=1}^{\ell} = \ \arg\max \quad & y_\ell^\top\, Q\, y_\ell \\ \text{s.t.} \quad & a_i \in \{0,1\} \\ & \textstyle\sum_{q=j}^{j+\bar{a}} a_q \le \bar{a}, \ \forall j \in \{1,\dots,\ell-\bar{a}\} \\ & \psi_0, \ \psi_{i+1} = [(1-a_i)\,\Phi_\text{h} + (a_i)\,\Phi_\text{m}]\,\psi_i \\ & \text{sequence randomness (Section 4.4).} \end{aligned} \tag{7}$$

The attacker maximises the impact on the output of the plant at the end of the attack sequence. The first constraint ensures that the chosen values of a_i are binary for the entire attack sequence. The second constraint guarantees that the system does not experience a shutdown due to the maximum consecutive number of deadline misses (following the specification given in Definition 4.1). The third constraint row in the optimisation problem introduces constraints that reformulate Equation (4) for all the attack iterations, specifying that the closed-loop state at the next time unit is either determined by Φ_h or Φ_m (Definitions 4.3–4.5).

Finally, we introduce a set of constraints that are meant to make the attack sequence appear *random* to the controller, and therefore avoid detection by an activity-based detection mechanism. We discuss these constraints in the following section.

4.4 Stealthiness

The attacker expects the controller to make an attempt to detect an ongoing attack. As stated in Section 3.2, we assume that the controller uses an activity-based detection mechanism that is based on checking $\mathcal{W}$ (Definition 3.2) and $\mathcal{M}$ (Definition 3.3). Given the shutdown mechanism in Definition 4.1, we assume that the controller records all possible long-term misses setting the cardinality of the set $\mathcal{M}$ to $\bar{a}+1$. Finally, the controller treats every deadline as an independent event and uses an estimate of the probability of missing a calculation deadline, possibly obtained using historical data or a machine learning model.

Definition 4.8 (Expected miss probability $\hat{p}_m$). $\hat{p}_\text{m}$ represents the expected probability that the controller experiences a deadline miss in absence of an attack.

We present two tests that the attack sequence should pass to appear random from the perspective of the controller. The first test

[2] Alternatively, one could formulate the optimisation problem to maximise the sum of the impact during the entire attack length.

is based on the binomial distribution and uses $\mathcal{W} = (w_i)_{i\in\{1,\dots,n_w\}}$. The second test is based on the geometric distribution and uses the historical data $\mathcal{M} = \{m_0, \dots, m_{\bar{a}}\}$.

4.4.1 Binomial distribution test. To check if the number of deadline misses in a window is justifiable given $\hat{p}_\mathrm{m}$ we use a one-tailed binomial test. In probability theory and statistics, the binomial distribution characterized by parameters n and p represents the discrete probability distribution of the number of successes in a series of n independent trials. Each trial poses a binary question with outcomes classified as success, with probability p, or failure, with probability $1-p$. For a sequence of binary events, such as deadline hits and misses, the binomial test offers a robust framework for finding the statistical significance that the distribution over the two categories deviates from the expected. For the test we define the null hypothesis H_0 and the alternative hypothesis H_1 for the expected probability $\hat{p}_\mathrm{m}$ of a deadline miss for the last n_w deadlines.

$$\begin{aligned} H_0 &: p \leq \hat{p}_\mathrm{m} \\ H_1 &: p > \hat{p}_\mathrm{m} \end{aligned} \tag{8}$$

If H_0 can be rejected, it is statistically significant (at the level α) that $p > \hat{p}_\mathrm{m}$ and a detector would raise an alarm that the system is under attack as the probability for a deadline miss occurs to be higher than nominal. On the other hand, if H_0 cannot be rejected, there is no statistical significance that the system is under attack, hence the attack would not be detected.

We introduce an additional constraint to the optimisation problem in Equation (7), ensuring that the null hypothesis H_0 cannot be rejected. To achieve this, we compute the likelihood of observing the actual outcome–or a more extreme one–under the assumption that the null hypothesis holds. If the probability of observing at least $\sum_i w_i$ deadline misses exceeds the significance level α, we cannot reject the null hypothesis. Rather than recalculating this probability for each iteration, we use binomial distribution quantiles as the constraint in the optimisation problem. The quantile provides the maximum number of allowable deadline misses in a sequence of observations, given the sequence length n_w, the expected probability $\hat{p}_\mathrm{m}$, and the significance level α.

This results in the following constraint:

$$\sum_i (w_i) \leq \mathfrak{B}_{1-\alpha}\,(n_w, \hat{p}_\mathrm{m}), \tag{9}$$

where $\mathfrak{B}_\gamma\,(n, p)$ denotes the quantile of order γ for the binomial distribution with parameters n and p.

4.4.2 Geometric distribution test. The geometric distribution is the discrete probability distribution that describes when the first success in an infinite sequence of independent and identically distributed Bernoulli trials occurs. The controller is expected to check that the computational delays follow a geometric distribution with failure probability $\hat{p}_\mathrm{m}$ and success probability $1-\hat{p}_\mathrm{m}$. Implementing this detector as an exact test, such as the binomial test, would require calculating the probability of all possible outcomes for the variable representing long-time counters, denoted here as $\mathcal{M}$. This calculation is computationally intensive, as it involves considering every possible combination of outcomes. To make the test manageable, we instead use an approximate method, the χ^2 test, which provides a close estimate without needing to calculate each individual outcome [17]. This approach simplifies the process and reduces the computational load.

To be undetected, the attacker ensures that the delays would not be detected using a χ^2 test that checks whether the sequence of successes and failures is compatible with said geometric distribution. The χ^2 test is a statistical method used to determine whether there is a significant difference between observed and expected frequencies in categorical data. In this context, the categories correspond to the different lengths of the delays, which are tracked by the long-term counters $\mathcal{M}$.

The expected frequency for category m_i is calculated according to the geometric distribution and is given by $E_i = \hat{p}_\mathrm{m}^i \cdot (1-\hat{p}_\mathrm{m}) \cdot N$, where $N = \sum_{i=0}^{\bar{a}} m_i$ is the total number of finished control tasks. For the final category $\bar{a}$ the expected frequency is $E_{\bar{a}} = 1 - \sum_{i=0}^{\bar{a}-1} E_i$, as longer delays are considered impossible and the total probability should equal 1. We test here the hypothesis $H_0 : p = \hat{p}_\mathrm{m}$ versus $H_1 : p \neq \hat{p}_\mathrm{m}$. When the difference between the observed and expected frequencies is large, then H_0 can be rejected. The difference is calculated as

$$S_{\chi^2} = \sum_{i=0}^{\bar{a}} \frac{(E_i - m_i)^2}{E_i}, \tag{10}$$

where m_i is the number of observed delays in category i (from Definition 3.3). For the attack to be stealthy, it should not be possible to reject H_0, therefore we require the sum to be smaller than the $1-\alpha$ quantile with $\bar{a}$ degrees of freedom for the χ^2 distribution. This gives,

$$S_{\chi^2} \leq \chi^2_{1-\alpha}(\bar{a}) \tag{11}$$

as an extra constraint for the optimisation problem in Equation (7).

5 EXPERIMENTAL RESULTS

The optimisation problem presented in Equation (7), including the stealthiness constraints from Section 4.4, is a Mixed Integer Quadratically Constrained Programming (MIQCP) problem. We implemented the problem in `gurobi` [9], using `gurobipy` and employing indicator constraints to enforce the closed-loop Equation (4).

In the following we present the results obtained with two different control systems with different open-loop stability properties. As a representative of a stable system that needs regulating to a given setpoint we select a quadruple-tank process [10] that has already been used in the past to illustrate security threats [35]. For the class of open-loop unstable systems, we instead use a Furuta pendulum [49], which should be regulated to zero, corresponding to the pendulum being in the upright position. In Section 5.1, we discuss the common part of the setup and configuration of the experiments. In Sections 5.2 and 5.3, we detail the specific process information. Finally, in Section 5.4 we draw some conclusions.

5.1 Experimental setup

In the experiments we present results obtained using the following parameters, selected to be realistic with respect to real-world control systems:

- maximum consecutive deadline misses $\bar{a} = \{1, \dots, 5\}$,
- expected miss probability $\hat{p}_\mathrm{m} = \{0.01, 0.03, \dots, 0.25\}$,
- sliding window length $n_w = \{10, 12, \dots 20\}$,

- significance level $\alpha = 0.01$.

The choice of parameter values was informed by practical considerations. The expected miss probability $\hat{p}_{\mathrm{m}}$ reflects empirical observations from industrial real-time systems where deadline misses occur sporadically under normal conditions. The window size n_w aligns with detection mechanisms commonly found in runtime monitors for CPS. The initial state x_0 is randomly generated from a uniform distribution over an interval in which we expect the system to be operating under nominal conditions with disturbances. The values of z_0 and u_0 are initialised to zero, while y_0 and $\tilde{y}_0$ are consistent with the randomised plant state. We investigate attack lengths $\ell = \{1, ..., 25\}$ for the optimisation problem to be tractable, showing that in practice even short attacks can cause significant problems in the system behaviour.

For each configuration, we simulate the system in the absence of attack and solve the optimisation problem in three different configurations: with the binomial distribution test alone, with the geometric distribution test alone, and with both combined. To evaluate the impact of the attack we calculate the attack effectiveness ω, defined as follows.

Definition 5.1 (Attack effectiveness ω). The attack effectiveness ω is the difference between the value of $y_\ell^\top Q\, y_\ell$ in the presence of the attack and in the absence of the attack, starting from the same initial conditions ψ_0.

5.2 Quadruple tank (open-loop stable)

The quadruple tank process is a well-known benchmark problem in control theory, that involves the control of the water levels in four interconnected tanks, placed in a 2×2 grid, using two pumps. Each pump distributes water into one of the top-level tanks and the bottom tank in the opposite column. The system state is the water level in the four tanks, while the output is the level of the bottom tanks. The linearized quadruple-tank model has a multivariable zero that can be placed either inside or outside of the unit disc by adjusting a valve. The location and direction of this zero are crucial for control design and have clear physical interpretations, making the system a classical control benchmark.[3] We here study a non-minimum phase configuration controlled by a Linear Quadratic Gaussian (LQG) regulator with a sampling period of 0.1 s that uses the hold actuation strategy. The following matrices describe the system and the controller:

$$A = \begin{bmatrix} 0.9957 & 0 & 0.0061 & 0 \\ 0 & 0.9957 & 0 & 0.0061 \\ 0 & 0 & 0.9939 & 0 \\ 0 & 0 & 0 & 0.9939 \end{bmatrix}, \quad B = \begin{bmatrix} 0.0098 & 6.976 \cdot 10^{-5} \\ 6.976 \cdot 10^{-5} & 0.0098 \\ 0 & 0.0228 \\ 0.0228 & 0 \end{bmatrix},$$

$$C = \begin{bmatrix} 0.5 & 0 & 0 & 0 \\ 0 & 0.5 & 0 & 0 \end{bmatrix}, \quad D = \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix},$$

$$F = \begin{bmatrix} 0.5998 & -0.0093 & 0.0118 & -0.0098 \\ -0.0093 & 0.5998 & -0.0098 & 0.0118 \\ -0.1800 & -0.0155 & 0.9709 & 0.0134 \\ -0.0155 & -0.1800 & 0.0134 & 0.9709 \end{bmatrix}, \quad G = \begin{bmatrix} 0.7729 & -0.0170 \\ -0.0170 & 0.7729 \\ 0.2776 & -0.0128 \\ -0.0128 & 0.2776 \end{bmatrix},$$

$$H = \begin{bmatrix} -0.6790 & -0.9424 & 0.5874 & -1.0097 \\ -0.9424 & -0.6790 & -1.0097 & 0.5874 \end{bmatrix}, \quad K = \begin{bmatrix} -0.5609 & -1.7327 \\ -1.7327 & -0.5609 \end{bmatrix}.$$

We select $Q = \mathrm{diag}\,\{1, 1\}$ assuming the attacker is equally interested in disrupting the water level in the two bottom tanks.

5.3 Furuta pendulum (open-loop unstable)

A Furuta pendulum, also known as a rotational inverted pendulum, is a mechanical system consisting of a rigid pendulum attached to a rotating base. The rotation is driven by a motor, with the goal to balance the pendulum in the upright position. The system is widely used in control theory and robotics as a benchmark problem to test algorithms for dynamic stabilisation. The system state is a three-dimensional vector that includes the pendulum angle, the pendulum angular velocity, and the angular velocity of the rotating arm. We experiment with a setup in which all the states are measurable, making the output vector the same as the system state. We use a Linear Quadratic Regulator (LQR) with a sampling period of 0.005 s, and test both the hold and the zero actuation strategies. We conducted experiments with real hardware, that can be checked in a video linked in the result description section. The following matrices describe the system model and controller:

$$A = \begin{bmatrix} 1.0026 & 0.0050 & 0 \\ 1.0399 & 1.0026 & 0 \\ -0.0675 & -0.0002 & 1 \end{bmatrix}, \quad B = \begin{bmatrix} -00.0843 \\ -33.7508 \\ 39.2131 \end{bmatrix}, \quad C = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix},$$

$$D = 0, \quad F = 0, \quad G = 0, \quad H = 0, \quad K = \begin{bmatrix} 0.4280 & 0.0307 & 0.0119 \end{bmatrix}.$$

We select $Q = \mathrm{diag}\,\{1, 0, 0\}$, reflecting the primary objective of the attacker of destabilising the pendulum.

5.4 Results

We here present selected results from the set of experiments executed with the two systems, showing the attack effectiveness ω. First, we investigate the effect of the parameter $\bar{a}$. Second, we compare the binomial test, the geometric test, and the combination of the two. Finally, we use the Furuta pendulum to show the effect of the window length n_w and a comparison between the zero and hold actuation strategies.

5.4.1 Effect of the maximum consecutive deadline misses $\bar{a}$. In our first set of experiments, we aim to assess whether the defence strategy that shuts the control system down after a set of consecutive deadline misses is effective against the computational delay attack presented in this paper.

Figure 2 shows the attack effectiveness ω for different values of $\bar{a}$ for both the quadruple tank and the Furuta pendulum plants when the window size for the detector $n_w = 20$ and the binomial and geometric tests are used in a combined fashion for $\hat{p}_{\mathrm{m}} \in \{0.09.0.17, 0.25\}$. For the Furuta pendulum, the zero actuation strategy is used when a deadline is missed.

It is possible to see that longer attacks do not necessarily imply higher effectiveness: in fact, ω reaches its upmost value with an attack length ℓ around 10 iterations. In particular, in the Furuta pendulum case, it is very obvious that there are two local maxima in the plotted curves. The first one is due to the effect of the

[3] Zeros in the unit disc are typically considered good for control, because they increase the closed-loop system robustness to disturbances and unmodelled dynamics. On the contrary, zeros outside of the unit disc are often associated with control challenges, as they lead to non-minimum phase behaviour. The control strategy then needs to take into account that the input changes (like increasing or decreasing the flow rate) may initially cause the opposite effect on the process than what is desired.

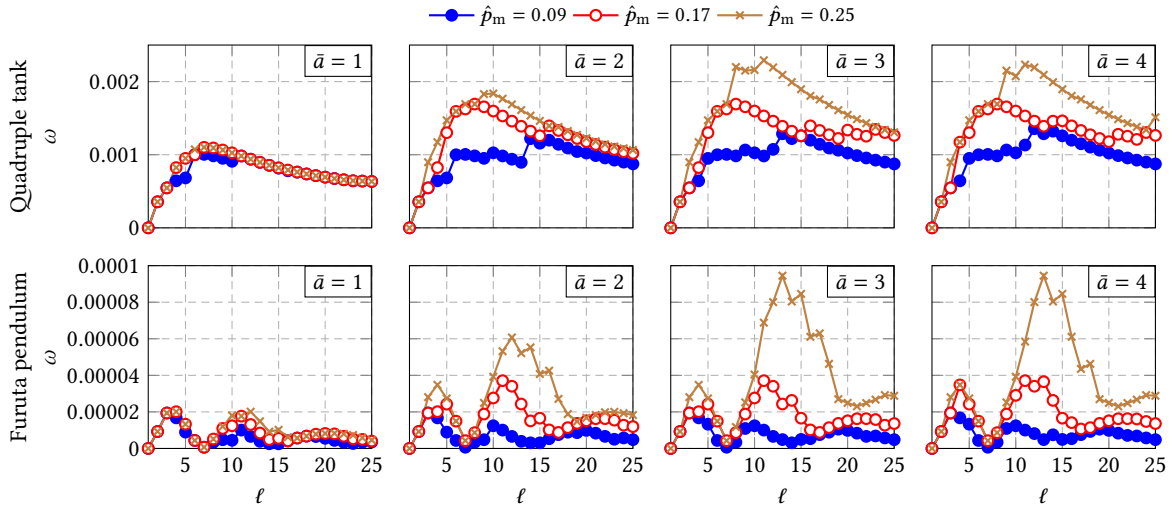


Figure 2: Attack effectiveness ω with window size $n_w = 20$ and combined binomial and χ^2 detector. The tests are conducted using the hold actuation policy for the quadruple tank and the zero actuation policy for the Furuta pendulum.

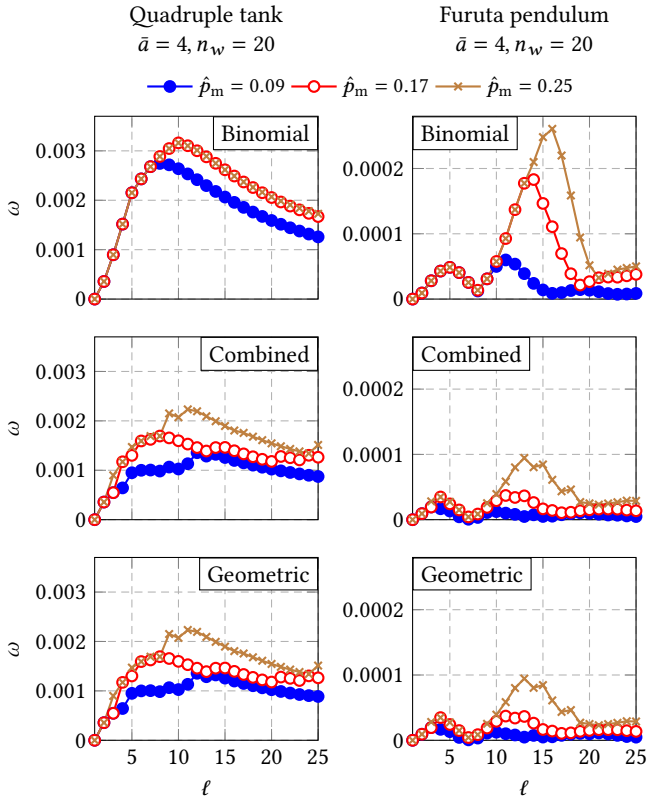


Figure 3: Attack effectiveness ω with different detection mechanisms: binomial test-based detector, geometric test-based detector and combined. The tests are conducted using the hold actuation policy for the quadruple tank and the zero actuation policy for the Furuta pendulum.

initial conditions: unsurprisingly, short attacks tend to be highly dependent on the initial conditions of the process; in this case, the solver tends to introduce all the possible artificial delays at the beginning of the computation, trying to take advantage of the inherent distance between the initial conditions and the control objectives. For longer attacks, this is no longer beneficial, as the budget of disruption that can be introduced is limited. In this case, concentrating the attack at the beginning of the attack sequence gives the controller longer time to recover. As an example, for the Furuta pendulum with $\bar{a} = 4$, $\hat{p}_m = 0.25$, $n_w = 20$ and the zero actuation strategy, the attack of $\ell = 5$ results in the sequence 11010, the attack of length $\ell = 15$ is 101011011101000, and when $\ell = 25$ we obtain 1100000000010110111011010. With longer attacks, the attacker has to take advantage of the physical properties of the plant, and decide when it is the best moment to launch the attack. This is very obvious in the sequence above when $\ell = 25$. Interestingly, this is not always at the end of the attack sequence, but the periods in which the controller is delayed are interleaved with successful executions. For example, in the sequence with $\ell = 15$ the highest concentration of deadline misses is in the middle of the attack, and the optimal value has the last three iterations being successful.

Moving from the leftmost to the rightmost plot, it is possible to see that the $\bar{a}$ parameter has an impact on the attack effectiveness only for low numbers. In fact, the difference between $\bar{a} = 3$ and $\bar{a} = 4$ is very limited. This tells us that the shutdown mechanism presented in Definition 4.1 is effective against the computational delay attack only if the system can be shutdown very soon after missing one or two deadlines. This is not really feasible for many industrial controllers, that are known to experience deadline misses in their regular operation [24, 47]. Even though the plots only show a limited set of results, the same considerations carry over to different values of the window size and of $\hat{p}_m$. As soon as the value of $\bar{a} \geq 3$, the effectiveness of the attack does not change

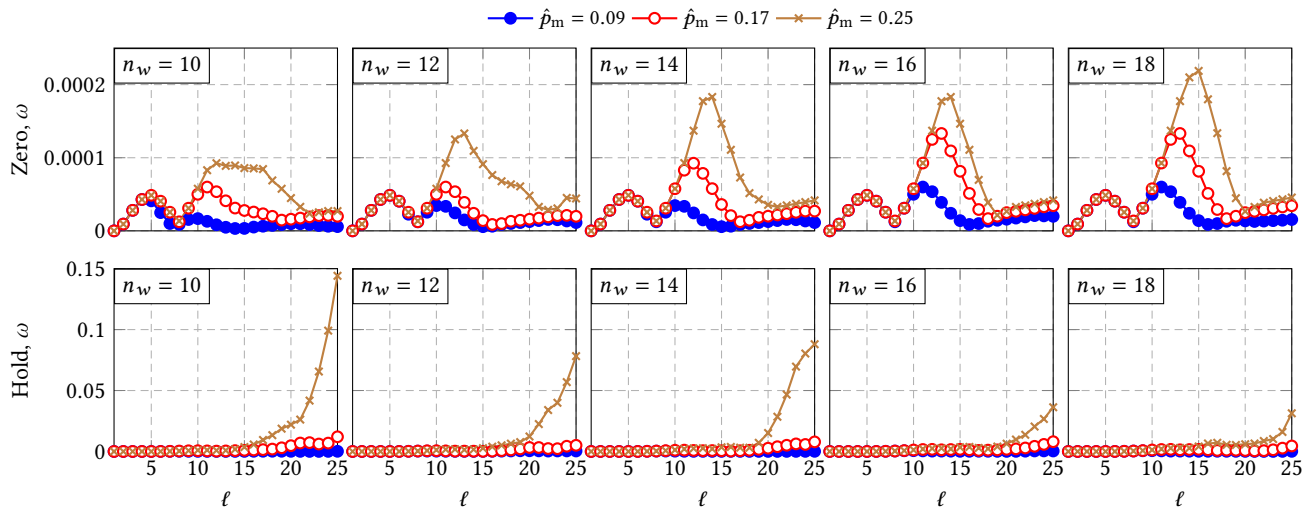


Figure 4: Attack effectiveness ω for the Furuta pendulum with zero and hold actuation strategy in case of a deadline miss, varying the window size n_w, with $\bar{a} = 4$ and using the binomial test-based attack detection mechanism.

significantly when the system could experience more consecutive deadline misses.

5.4.2 Test effectiveness. In the second set of experiments, we discuss the ability of the statistical tests to limit the attack effectiveness.

Figure 3 shows the attack effectiveness for the three detection types: (i) binomial test alone, (ii) combined binomial and geometric test, and (iii) geometric test alone. The left column shows the results obtained with the quadruple tank plant, while the right column shows the Furuta pendulum that uses the zero actuation policy. Lower values indicate that the detection mechanism is more effective in limiting the ability of the attacker to interfere with the physical plant. In fact, it is possible to see that the geometric test with the χ^2 detector dominates the binomial test. This is generally true also for other combinations of $\hat{p}_m$, $\bar{a}$, n_w, and α, with few exceptions. According to these results, testing based on long-term information that captures rare events is enough to protect the control system as much as possible and the binomial test, that takes into account only the last iterations in a window seems to be rather ineffective in limiting the attacker.

5.4.3 Effect of the window length n_w and zero-hold comparison. This last set of experiments has two different purposes. The first is showing the effect of an increase in the window length for the binomial test-based detector. The second is to compare the zero and hold actuation strategies for the Furuta pendulum. Figure 4 shows the results for windows of 10, 12, 14, 16, and 18 iterations. The first row shows the attack effectiveness when the zero strategy is adopted, while the second row shows the hold strategy. The first obvious remark is that with the hold strategy, the attack is actually able to destabilise the system, provided that the estimated probability of deadline miss is high enough (see the plots for $\hat{p}_m = 0.25$ in the second row).[4] For the Furuta pendulum, the zero strategy is much better than the hold strategy as a defence mechanism. The second remark is that the window length has a different impact on the attack effectiveness depending on which actuation strategy is used. For the hold strategy, shorter detection windows boost the effectiveness of the attack. In fact, a shorter window allows the attacker to introduce, on average, a higher number of deadline misses. A longer window, on the contrary, introduces the possibility of longer attack bursts, yet decreases the average number of misses introduced. We illustrate this properly by showing the attacks calculated for hold, $\ell = 25$, and $n_w \in \{10, 12, 14\}$ in Table 1.

Table 1: Attack sequences calculated for the Furuta pendulum with the hold actuation strategy, $\hat{p}_m = 0.25$, $\bar{a} = 4$, $\ell = 25$, and different window sizes n_w, including an indication of the percentage of deadline misses introduced in a single window.

n_w	Attack	% in $\mathcal{W}$	$\sum_i (a_i)$
10	1110111000111000111011100	60%	15
12	1011110001101000111101100	58%	14
14	1101111000110001111011000	57%	14

The percentage of misses introduced in a single window decreases (the optimisation problem allows for 7 misses in a window of 12 deadlines, and 8 misses in a window of 14 deadlines). This experiment shows that the window length of the binomial detection test should be carefully tuned to the specific detection needs.

6 CONCLUSION

This paper analyses a stealthy computational delay attack on control signal timing in CPSs, demonstrating how attackers can subtly disrupt system performance without detection by framing the attack as an optimisation problem. Even minor, well-timed computational delays can destabilise physical systems while remaining undetected.

Experimental results show that shutdown mechanisms are ineffective against informed attackers that avoid causing consecutive

[4] Selected experiments can be seen in the video: https://youtu.be/L15ab0cy-7Q.

deadline misses. Furthermore, the attack's impact varies with the actuation strategy, highlighting that plant dynamics influence vulnerability. These insights indicate that conventional CPS defences are insufficient against adaptive stealthy attacks.

REFERENCES

[1] S. Amin, A.A. Cárdenas, and S.S. Sastry. 2009. Safe and Secure Networked Control Systems under Denial-of-Service Attacks. In *Hybrid Systems: Computation and Control*. Springer Berlin Heidelberg, 31–45.

[2] S.K. Baruah, P. Ekberg, M. Hosseinzadeh, A. Li, B.C. Ward, and N. Zhang. 2023. Who's Afraid of Butterflies? A Close Examination of the Butterfly Attack. In *Real-Time Systems Symposium*. IEEE, 53–63. https://doi.org/10.1109/RTSS59052.2023.00015

[3] G. Bianchin and F. Pasqualetti. 2018. Time-Delay Attacks in Network Systems. In *Cyber-Physical Systems Security*. Springer, 157–174. https://doi.org/10.1007/978-3-319-98935-8_8

[4] A. Cetinkaya, H. Ishii, and T. Hayakawa. 2019. An Overview on Denial-of-Service Attacks in Control Systems: Attack Models and Security Analyses. *Entropy* 21, 2 (2019). https://doi.org/10.3390/e21020210

[5] M.S. Chong, H. Sandberg, and A.M.H. Teixeira. 2019. A Tutorial Introduction to Security and Privacy for Cyber-Physical Systems. In *18th European Control Conference*. IEEE, 968–978. https://doi.org/10.23919/ECC.2019.8795652

[6] C. De Persis and P. Tesi. 2015. Input-to-State Stabilizing Control Under Denial-of-Service. *IEEE Trans. Automat. Control* 60, 11 (2015), 2930–2944. https://doi.org/10.1109/TAC.2015.2416924

[7] A. Erba, R. Taormina, S. Galelli, M. Pogliani, M. Carminati, S. Zanero, and N.O. Tippenhauer. 2020. Constrained Concealment Attacks against Reconstruction-based Anomaly Detectors in Industrial Control Systems. In *Computer Security Applications Conference*. ACM, 480–495. https://doi.org/10.1145/3427228.3427660

[8] M. Gunnarsson, N. Vreman, and M. Maggio. 2023. Trusted Execution of Periodic Tasks for Embedded Systems. In *IFAC World Conference*, Vol. 56–2. IFAC, 8845–8850. https://doi.org/10.1016/j.ifacol.2023.10.082

[9] Gurobi. 2024. Optimizer Reference Manual. https://www.gurobi.com

[10] K.H. Johansson. 2000. The quadruple-tank process: a multivariable laboratory process with an adjustable zero. *IEEE Transactions on Control Systems Technology* 8, 3 (2000), 456–465. https://doi.org/10.1109/87.845876

[11] A. Jokic, A. Khazraei, M. Petrovic, Z. Jakovljevic, and M. Pajic. 2023. Cyber-Attacks on Wheeled Mobile Robotic Systems with Visual Servoing Control. In *IROS*. IEEE, 6342–6348. https://doi.org/10.1109/IROS55552.2023.10341376

[12] A. Khazraei, H. Meng, and M. Pajic. 2023. Stealthy Perception-based Attacks on Unmanned Aerial Vehicles. In *International Conference on Robotics and Automation*. IEEE, 3346–3352. https://doi.org/10.1109/ICRA48891.2023.10160900

[13] A. Khazraei and M. Pajic. 2022. Attack-resilient state estimation with intermittent data authentication. *Automatica* 138 (2022).

[14] A. Khazraei and M. Pajic. 2022. Resiliency of Nonlinear Control Systems to Stealthy Sensor Attacks. In *61st IEEE Conference on Decision and Control*. IEEE, 7109–7114. https://doi.org/10.1109/CDC51059.2022.9992988

[15] J. Kim, C. Lee, H. Shim, Y. Eun, and J.H. Seo. 2016. Detection of sensor attack and resilient state estimation for uniformly observable nonlinear systems. In *55th Conference on Decision and Control*. IEEE, 1297–1302. https://doi.org/10.1109/CDC.2016.7798445

[16] E. Korkmaz, A. Dolgikh, M. Davis, and V. Skormin. 2016. ICS security testbed with delay attack case study. In *Military Communications Conference*. IEEE, 283–288. https://doi.org/10.1109/MILCOM.2016.7795340

[17] E.L. Lehmann and Joseph P. Romano. [n. d.]. *Testing Statistical Hypotheses*. Springer Cham. https://doi.org/10.1007/978-3-030-70578-7

[18] W.S. Levine. 2017. *The Control Handbook: Control System Fundamentals, Second Edition*. CRC Press, USA.

[19] A. Li, H. Liu, J. Wang, and N. Zhang. 2022. From Timing Variations to Performance Degradation: Understanding and Mitigating the Impact of Software Execution Timing in SLAM. In *International Conference on Intelligent Robots and Systems*. IEEE, 13308–13315. https://doi.org/10.1109/IROS47612.2022.9981275

[20] A. Li, J. Wang, S.K. Baruah, B. Sinopoli, and N. Zhang. 2024. An Empirical Study of Performance Interference: Timing Violation Patterns and Impacts. In *Real-Time and Embedded Technology and Applications Symposium*. IEEE, 320–333. https://doi.org/10.1109/RTAS61025.2024.00033

[21] A. Li, J. Wang, and N. Zhang. 2021. Chronos: Timing Interference as a New Attack Vector on Autonomous Cyber-physical Systems. In *Conference on Computer and Communications Security*. ACM. https://doi.org/10.1145/3460120.3485350

[22] X. Lou, C. Tran, R. Tan, D.K.Y. Yau, and Z.T. Kalbarczyk. 2019. Assessing and mitigating impact of time delay attack: a case study for power grid frequency control. In *International Conference on Cyber-Physical Systems*. ACM, 207–216. https://doi.org/10.1145/3302509.3311042

[23] F. Maggi, S. Zanero, and V. Iozzo. 2008. Seeing the invisible: forensic uses of anomaly detection and machine learning. *SIGOPS Operating Systems Review* 42, 3 (2008), 51–58. https://doi.org/10.1145/1368506.1368514

[24] M. Maggio, A. Hamann, E. Mayer-John, and D. Ziegenbein. 2020. Control-System Stability Under Consecutive Deadline Misses Constraints. In *32nd Euromicro Conference on Real-Time Systems (LIPIcs, Vol. 165)*. Leibniz-Zentrum für Informatik, 21:1–21:24. https://doi.org/10.4230/LIPICS.ECRTS.2020.21

[25] R. Mahfouzi, A. Aminifar, S. Samii, M. Payer, P. Eles, and Z. Peng. 2019. Butterfly Attack: Adversarial Manipulation of Temporal Properties of Cyber-Physical Systems. In *Real-Time Systems Symposium*. IEEE, 93–106. https://doi.org/10.1109/RTSS46320.2019.00019

[26] F. Miao, Q. Zhu, M. Pajic, and G.J. Pappas. 2014. Coding sensor outputs for injection attacks detection. In *IEEE Conference on Decision and Control*. IEEE, 5776–5781. https://doi.org/10.1109/CDC.2014.7040293

[27] Y. Mo and B. Sinopoli. 2009. Secure control against replay attacks. In *47th Annual Allerton Conference on Communication, Control, and Computing*. IEEE, 911–918. https://doi.org/10.1109/ALLERTON.2009.5394956

[28] Y. Mo and B. Sinopoli. 2010. False data injection attacks in control systems. *Preprints of the 1st Workshop on Secure Control Systems* 1 (2010), 1–1.

[29] F. Pasqualetti, F. Dörfler, and F. Bullo. 2013. Attack Detection and Identification in Cyber-Physical Systems. *IEEE Trans. Automat. Control* 58, 11 (2013), 2715–2729. https://doi.org/10.1109/TAC.2013.2266831

[30] P. Pazzaglia, A. Hamann, D. Ziegenbein, and M. Maggio. 2021. Adaptive Design of Real-Time Control Systems subject to Sporadic Overruns. In *Design, Automation & Test in Europe Conference & Exhibition*. IEEE, 1887–1892. https://doi.org/10.23919/DATE51398.2021.9473913

[31] R. Romagnoli, B.H. Krogh, and B. Sinopoli. 2019. Design of Software Rejuvenation for CPS Security Using Invariant Sets. In *2019 American Control Conference*. IEEE, 3740–3745. https://doi.org/10.23919/ACC.2019.8815155

[32] H. Sandberg. 2021. *Stealthy False Data Injection Attacks in Feedback Systems Revisited*. Springer, 61–78. https://doi.org/10.1007/978-3-030-65048-3_4

[33] L. Schenato. 2009. To Zero or to Hold Control Inputs With Lossy Links? *IEEE Trans. Automat. Control* 54, 5 (2009). https://doi.org/10.1109/TAC.2008.2010999

[34] R.S. Smith. 2011. A Decoupled Feedback Structure for Covertly Appropriating Networked Control Systems. In *18th IFAC World Congress*, Vol. 44. IFAC, 90–95. https://doi.org/10.3182/20110828-6-IT-1002.01721

[35] A.M.H. Teixeira, D. Pérez, H. Sandberg, and K.H. Johansson. 2012. Attack models and scenarios for networked control systems. In *Conference on High Confidence Networked Systems*. ACM, 55–64. https://doi.org/10.1145/2185505.2185515

[36] A.M.H. Teixeira, H. Sandberg, and K.H. Johansson. 2015. Strategic stealthy attacks: The output-to-output ℓ2-gain. In *2015 54th IEEE Conference on Decision and Control*. IEEE, 2582–2587. https://doi.org/10.1109/CDC.2015.7402605

[37] A.M.H. Teixeira, I. Shames, H. Sandberg, and K.H. Johansson. 2012. Revealing stealthy attacks in control systems. In *Conference on Communication, Control, and Computing*. IEEE, 1806–1813. https://doi.org/10.1109/Allerton.2012.6483441

[38] A.M.H. Teixeira, I. Shames, H. Sandberg, and K.H. Johansson. 2015. A secure control framework for resource-limited adversaries. *Automatica* 51 (2015), 135–148. https://doi.org/10.1016/j.Automatica.2014.10.067

[39] A.M.H. Teixeira, K.C. Sou, H. Sandberg, and K.H. Johansson. 2015. Secure Control Systems: A Quantitative Risk Management Approach. *IEEE Control Systems Magazine* 35, 1 (2015), 24–45. https://doi.org/10.1109/MCS.2014.2364709

[40] D.I. Urbina, J.A. Giraldo, A.A. Cardenas, N.O. Tippenhauer, J. Valente, M. Faisal, J. Ruths, R. Candell, and H. Sandberg. 2016. Limiting the Impact of Stealthy Attacks on Industrial Control Systems. In *ACM Conference on Computer and Communications Security*. ACM, 1–1. https://doi.org/10.1145/2976749.2978388

[41] T. Wigren and A.M.H. Teixeira. 2023. Feedback Path Delay Attacks and Detection. In *IEEE Conference on Decision and Control*. IEEE, 3864–3871.

[42] T. Wigren and A.M.H. Teixeira. 2024. Delay Attack and Detection in Feedback Linearized Control Systems. In *European Control Conference*. IEEE, 1576–1583. https://doi.org/10.23919/ECC64448.2024.10590885

[43] W. Xing, X. Zhao, N. Zhao, G. Zong, B. Niu, and N. Xu. 2023. Denial-of-service attacks on cyber–physical systems with two-hop networks under round-robin protocol. *Systems & Control Letters* 180 (2023).

[44] W. Zhai, L. Liu, Y. Ding, S. Sun, and Y. Gu. 2023. ETD: An Efficient Time Delay Attack Detection Framework for UAV Networks. *Trans. Info. For. Sec.* 18 (Jan. 2023), 2913––2928. https://doi.org/10.1109/TIFS.2023.3272862

[45] Z. Zhang, S. Huang, F. Liu, and S. Mei. 2020. Pattern Analysis of Topological Attacks in Cyber-Physical Power Systems Considering Cascading Outages. *IEEE Access* 8 (2020), 134257–134267. https://doi.org/10.1109/ACCESS.2020.3006555

[46] X. Zhou, B. Ahmed, J.H. Aylor, P. Asare, and H. Alemzadeh. 2024. Hybrid Knowledge and Data Driven Synthesis of Runtime Monitors for Cyber-Physical Systems. *IEEE Trans. Dependable Secur. Comput.* 21, 1 (Jan. 2024), 12––30. https://doi.org/10.1109/TDSC.2023.3242653

[47] B. Åkesson, M. Nasri, G. Nelissen, S. Altmeyer, and R.I. Davis. 2020. An Empirical Survey-based Study into Industry Practice in Real-time Systems. In *Real-Time Systems Symposium*. IEEE, 3–11. https://doi.org/10.1109/RTSS49844.2020.00012

[48] K.J. Åstrom and R.M. Murray. 2008. *Feedback Systems: An Introduction for Scientists and Engineers*. Princeton University Press, USA.

[49] K.J. Åström and K. Furuta. 2000. Swinging up a pendulum by energy control. *Automatica* 36, 2 (2000), 287–295. https://doi.org/10.1016/S0005-1098(99)00140-5

ACKNOWLEDGEMENTS

Talitha Nauta and Martina Maggio are members of the ELLIIT Strategic Research Area at Lund University. Henrik Sandberg is a member of Digital Futures.

This work was partially supported by the Wallenberg AI, Autonomous Systems and Software Program (WASP) funded by the Knut and Alice Wallenberg Foundation via the NEST project DYNACON. For further information, see https://wasp-sweden.org/nest-project-dynacon/ and https://people.kth.se/~hsan/dynacon/.

A REPEATABILITY EVALUATION PACKAGE

A Repeatability Evaluation Package (REP) is available and permanently archived for this paper. The package contains the code to generate the data for Figures 2–4 and Table 1 (and much more data that we used to elaborate some of the statements in Section 5).

A.1 Artifact

The artifact can be downloaded from the following link.

https://doi.org/10.5281/zenodo.14792914

A.2 Requirements

The code is written in `python` (specifically `python3`) and uses the `gurobi` solver and the associated library `gurobipy`, together with libraries `numpy` and `scipy`. Specifically, the code has been tested with the following versions:

- `python3` 3.12.2
- `gurobipy` 11.0.1
- `numpy` 1.26.4
- `scipy` 1.12.0

under the operating systems MacOS, Linux, and Windows. To run the code it is necessary to obtain a `gurobi` license, following the instructions that can be found at https://www.gurobi.com/features/academic-named-user-license/. Without a license, it will be possible to replicate results with $\ell \lesssim 10$.

Finally, to plot the data, one needs to install a full latex distribution (including packages such as `tikz` and `pgfplots`). For Mac OS, this can be done installing `mactex` following the instructions from https://www.tug.org/mactex/mactex-download.html. The procedure to follow on Linux-based operating systems depends on the specific distribution. For Debian-based distributions (including Ubuntu), it is sufficient to run

```
> sudo aptitude install texlive-full
```

and for RedHat-based distributions (including Fedora), one can run

```
> sudo dnf install texlive-scheme-full
```

to have a complete installation. For Windows user, it is sufficient to install `texlive` following the instructions available at https://www.tug.org/texlive/windows.html.

A.3 Installation

The recommended way to execute the code is to define a virtual environment. This is accomplished using a terminal, navigating to the replication package folder, and giving the following sequence of commands:

```
> python3 -m venv .
> source ./bin/activate
> python3 -m pip install gurobipy numpy scipy
```

Note that the virtual environment installs the library `gurobipy`, but to solve reasonably-sized problems it is necessary to obtain a `gurobi` license, as indicated above.

A.4 Replication

Everything that follows should be executed in the virtual environment. If you are restarting your terminal window, you need to activate the virtual environment again, as done in the installation step (`source ./bin/activate`).

A.4.1 Test the environment. To test that the environment is installed correctly, you can simply run the command:

```
> python3 main_short.py
> latexmk -lualatex -shell-escape plot_short.tex
```

which is supposed to take about 2 hours to run on a reasonably recent machine (it only generates one of the lines in the figures). In case you experience errors with longer attack sequences (above 10), this usually means that your `gurobi` license is not recognised or installed correctly. We refer you to the `gurobi` installation troubleshooting in case you encounter said problem.

A.4.2 Figure 2. In order to replicate the results presented in Figure 2, give the following commands.

```
> python3 main_fig2.py
> latexmk -lualatex -shell-escape plot_fig2.tex
```

The first command is expected to take a couple of days to complete.

A.4.3 Figure 3. In order to replicate the results presented in Figure 3, give the following commands.

```
> python3 main_fig3.py
> latexmk -lualatex -shell-escape plot_fig3.tex
```

The first command is expected to take a couple of days to complete.

A.4.4 Figure 4. In order to replicate the results presented in Figure 4, give the following commands.

```
> python3 main_fig4.py
> latexmk -lualatex -shell-escape plot_fig4.tex
```

The first command is expected to take a couple of days to complete.

A.4.5 Complete replication. In order to generate the entire set of data to replicate all the figures and the additional data that we used for the conclusions presented in the results section you can simply give the following command.

```
> python3 main.py
```

This command is expected to take several weeks to complete (note that it generates more data compared to the data used for Figures 2–4 and Table 1). You can then generate the figures using the `latexmk` commands above. More detailed instructions can be found in the provided file `README.md`.

Remark: Note that the running time increases with the sequence length ℓ. Before the code does terminates its execution, it is already possible to check the results for shorter sequences.